

Sim-to-Real Humanoid Stand-and-Walk Control via Reinforcement Learning

Junlei Zhu, Jingwei Peng, Yansong Bai
{zhujl12024, pengjw2024, baiys2022}@shanghaitech.edu.cn

May 2025

1 Introduction

Humanoid robots hold great potential, but their locomotion control remains a complex challenge. Developing a generalizable method for performing common tasks is still difficult, and even executing a single task often involves concerns related to safety and performance. Although many investigations have been conducted in this area [1], further work is needed to achieve practical solutions.

This project proposes the use of Proximal Policy Optimization (PPO) [2, 6], a state-of-the-art reinforcement learning algorithm, to address the motion control problems inherent in humanoid robotics. PPO is renowned for its ability to deliver stable and sample-efficient policy updates, making it particularly well-suited for complex control tasks where traditional methods often struggle with instability and convergence issues. By leveraging PPO, this project aims to develop a robust framework that can learn adaptable control strategies to manage the intricate balance and coordination required for humanoid locomotion.

In this project, we aim to implement PPO on the Dora2 [5] platform. As robotics continues to advance, enabling humanoid robots to perform autonomous locomotion has become a key research focus. We propose to explore reinforcement learning (RL) techniques to train a humanoid robot to stand and walk autonomously in simulation and real world. Our objective is to develop a system that can adapt its gait and maintain balance during walking, leading to more efficient and natural locomotion.

2 Problem formulation

We model the autonomous walking task as a Partially Observable Markov Decision Process (POMDP), as the robot does not have access to the full state of the environment at all times due to sensor noise, occlusion, or limited perception. Formally, a POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, R, \Omega, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ the set of actions, $\mathcal{O}$ the set of observations, T the transition function, R the reward function, Ω the observation function, and γ the discount factor. The agent receives an observation $o_t \in \mathcal{O}$ at each timestep and selects an action $a_t \in \mathcal{A}$ to interact with the environment.

A more detailed representation is $\mathcal{S} = \{s = (q, \dot{q}) \mid q \in [q_{\min}, q_{\max}], \dot{q} \in [\dot{q}_{\min}, \dot{q}_{\max}]\}$ and $\mathcal{A} = \{a = \tau \mid \tau \in [\tau_{\min}, \tau_{\max}]\}$. For transition function:

$$p(s, s', a) = f_{S_t, S_{t+1}}(S_{t+1} = s' \mid S_t = s, A_t = a)$$

Where q and $\dot{q}$ are configuration and joint velocity of robot respectively. τ is torque of robot joint motor. These quantities are range-bound and continuous.

In the process of simulation, In a fixed scene, knowing the position, velocity and a given torque gives the next moment state, so it has markov property.

3 PPO-Based Humanoid Locomotion Training Framework

We adopt the Proximal Policy Optimization (PPO) algorithm [6] due to its robustness and efficiency in continuous control settings. PPO is a policy gradient method that maximizes a clipped surrogate objective, effectively preventing excessively large updates that could destabilize training—an essential feature when dealing with the high-dimensional action space of humanoid robots.

The policy $\pi_\theta(a_t \mid s_t)$ and value function $V_\phi(s_t)$ are both parameterized by neural networks. These networks are trained using trajectory rollouts collected in a simulated environment. PPO ensures stable learning by balancing policy improvement and proximity to the previous policy.

An overview of the training pipeline is illustrated in Fig. 1. The overall PPO loss function augmented with an auxiliary estimation loss is defined as:

$$\mathcal{L}^{\text{PPO}}(\theta, \phi) = \mathbb{E}_t [\mathcal{L}^{\text{clip}}(\theta) - c_1 \cdot \mathcal{L}^{\text{value}}(\phi) + c_2 \cdot \mathcal{L}^{\text{entropy}}(\theta) + c_3 \cdot \mathcal{L}^{\text{estimator}}(\theta)] \quad (1)$$

- **Clipped policy loss:**

$$\mathcal{L}^{\text{clip}}(\theta) = -\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ is the probability ratio, and $\hat{A}_t$ is the estimated advantage. Maximizes policy improvement while constraining updates via clipping to avoid instability.

- **Value function loss:**

$$\mathcal{L}^{\text{value}}(\phi) = - (V_\phi(s_t) - \hat{R}_t)^2$$

where $V_\phi(s_t)$ is the predicted value and $\hat{R}_t$ is the target return. Trains the value network V_ϕ to accurately estimate state values, reducing policy update variance.

- **Entropy bonus:**

$$\mathcal{L}^{\text{entropy}}(\theta) = -\mathcal{H}[\pi_\theta(\cdot \mid s_t)] = -\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)$$

which encourages exploration by maximizing the policy entropy.

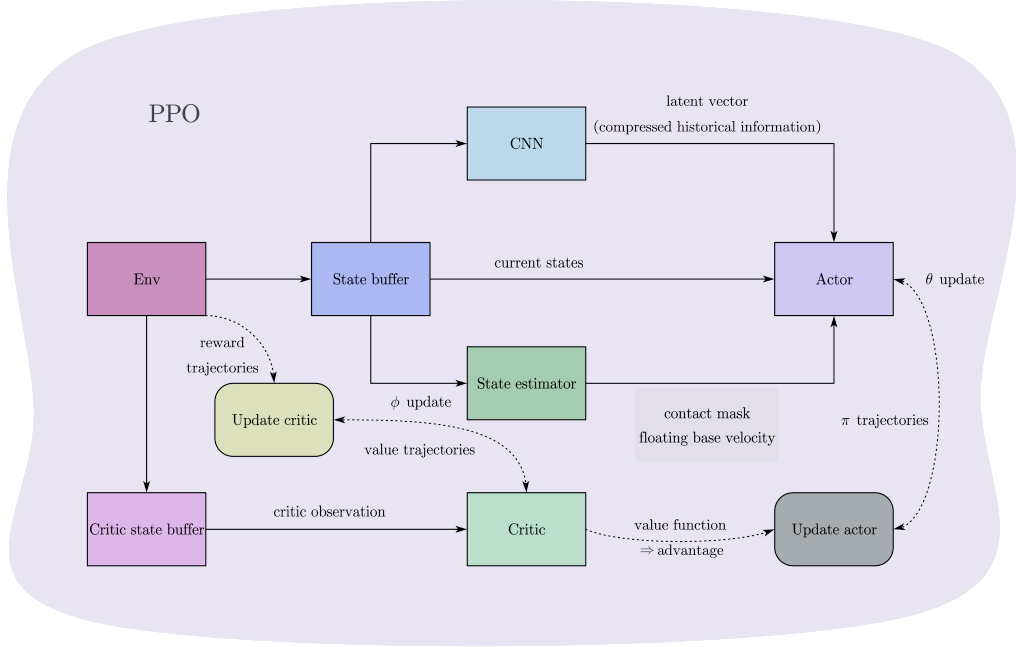


Figure 1: Reinforcement learning block diagram for the Dora2 robot locomotion task.

- **Auxiliary estimator loss:**

$$\mathcal{L}^{\text{estimator}}(\theta) = \|\hat{v}_{\text{base}} - v_{\text{gt}}\|^2 + \text{BCE}(\hat{c}, c_{\text{gt}})$$

where $\hat{v}_{\text{base}}$ is the predicted base velocity, v_{gt} is the ground truth, $\hat{c}$ is the predicted contact label, and BCE denotes the binary cross-entropy loss. Optional auxiliary loss (not in original PPO). Used for regularization, representation learning, or model-based tasks.

3.1 Observer-Augmented PPO Architecture

To tackle these challenges, we adopt an Asymmetric-Observation PPO framework enhanced with an observer module. The architecture is organized as follows:

We design an asymmetric actor-critic architecture in which the actor and critic operate over different observation spaces. The actor relies solely on onboard and estimated signals, whereas the critic receives additional privileged information during training to facilitate more accurate value estimation.

3.1.1 Actor Observations

The actor policy is conditioned on a combination of proprioceptive measurements, estimated state variables, command inputs, and temporally encoded features. Specifically, the observation vector includes:

- Proprioceptive sensing: joint positions and joint velocities, each in $\mathbb{R}^{12}$, along with IMU-based orientation (pitch and yaw) and angular velocity, yielding a total of $\mathbb{R}^5$.
- Command inputs ($\mathbb{R}^5$): target linear velocities in the x and y directions, a yaw rate command, and a locomotion phase signal that modulates cyclic behavior.
- Estimated state information ($\mathbb{R}^5$): includes an estimated floating base linear velocity ($\mathbb{R}^3$) and binary foot contact indicators for both feet ($\mathbb{R}^2$), provided by an auxiliary observation network.
- Temporal context encoding: a convolutional encoder, denoted $h_t = \text{CNN}(o_{t-k:t})$, captures recent temporal dynamics over a fixed-length observation window, enabling the policy to account for motion trends and contact history.

This observation formulation ensures that the actor operates under realistic sensory limitations, mirroring what is typically available on physical hardware.

3.1.2 Privileged Critic Observations

During training, the critic is granted access to privileged information that is not available to the actor at deployment time. This asymmetry improves learning efficiency and value estimation accuracy. The privileged observation space includes:

- Command input ($\mathbb{R}^5$): identical to that of the actor.
- Normalized joint positions ($\mathbb{R}^{12}$): difference between actual joint positions and a default pose, scaled by a position normalization factor.
- Normalized joint velocities ($\mathbb{R}^{12}$): joint angular velocities scaled for numerical stability.
- Current and delta actions ($\mathbb{R}^{12}$ each): applied motor commands and their first-order differences, capturing control history and actuation dynamics.
- Floating base kinematics: Linear velocity ($\mathbb{R}^3$), angular velocity ($\mathbb{R}^3$), and orientation in Euler angles ($\mathbb{R}^3$), all based on ground-truth simulation data and appropriately scaled.
- External perturbations: random external forces ($\mathbb{R}^2$) and torques ($\mathbb{R}^3$) applied to the robot base to promote robustness.
- Environmental parameters: Friction coefficient of the terrain ($\mathbb{R}^1$). Scaled body mass ($\mathbb{R}^1$) for simulating variability in payload or model uncertainties.
- Contact-related features: Stance mask ($\mathbb{R}^2$): expected foot-ground contact derived from phase-based gait scheduling. Contact mask ($\mathbb{R}^2$): binary indicators of actual contact derived from physics simulation.

This privileged observation space leverages full-state and contact feedback from the simulator, which, while unavailable during deployment, significantly accelerates policy training by stabilizing critic supervision.

3.2 Domain Randomization

To improve the robustness and transferability of the learned policy from simulation to the real world, we incorporate domain randomization during training. This technique introduces controlled variability in the simulation to prepare the policy for real-world uncertainties. Specifically, we randomize the following factors across training episodes:

- **Physical properties:** such as link masses, joint damping coefficients, and surface friction.
- **Sensor noise and latency:** Gaussian noise is added to observations, and delays are introduced to simulate real-world sensing imperfections.
- **External disturbances:** such as mild pushes, force perturbations, and uneven terrain features to mimic real-world disturbances.

This strategy enables the learned policy to generalize better across different simulation conditions, thereby increasing the likelihood of successful deployment on real hardware.

3.3 Reward Design

The reward function is designed to encourage stable, energy-efficient, and natural humanoid locomotion. We incorporate multiple reward terms that each target specific aspects of motion quality:

- **Forward Progress:** A positive reward is given for moving forward in the commanded direction.
- **Energy Efficiency:** Penalties are applied for excessive torque, velocity, or acceleration to promote efficient motor usage.
- **Stability:** Rewards are given for maintaining balance, while penalties are applied for falling or unstable behavior such as high base acceleration.
- **Smoothness:** Sudden changes in joint angles or control actions are penalized to produce more natural and smoother movement.

To better structure the reward space, we define a comprehensive set of reward components summarized in Table 1. These components are empirically weighted and combined to form the total reward used during training.

3.4 Sim-to-Real Transfer Strategy

To enhance the robustness and transferability of our control policy, we apply domain randomization during training. Random variations in physical parameters—including mass, joint friction, actuation latency, contact dynamics, and terrain profiles—are introduced in Isaac Gym to encourage the emergence of a policy that generalizes well across diverse scenarios.

Table 1: Reward function definitions

Reward Name	Expression
Joint Position Alignment	$r = \exp(-2\ \Delta q\) - 0.2 \cdot \text{clamp}(\ \Delta q\ , 0, 0.5)$
Feet Distance	$r = \frac{1}{2} [\exp(-100 d_{\min}) + \exp(-100 d_{\max})]$
Knee Distance	$r = \frac{1}{2} [\exp(-120 d_{\min}) + \exp(-100 d_{\max})]$
Foot Slip	$r = \sum \sqrt{\ \mathbf{v}_{\text{foot}}\ } \cdot \mathbb{I}_{\text{contact}}$
Feet Air Time	$r = \sum \text{clamp}(t_{\text{air}}, 0, 0.5) \cdot \mathbb{I}_{\text{first contact}}$
Gait Phase Matching	$r = \text{mean}(\mathbb{I}_{\text{contact}=\text{phase}})$
Base Orientation	$r = \frac{1}{2} [\exp(-10 \cdot \ \phi_{xy}\ _1) + \exp(-20 \cdot \ \mathbf{g}_{xy}\)]$
Feet Contact Force	$r = \sum \text{clamp}(\ \mathbf{F}\ - F_{\max}, 0, 250)$
Default Joint Position	$r = \exp(-100 \cdot \delta_{\text{yaw/roll}}) - 0.01 \cdot \ \Delta q\ $
Base Height	$r = \exp(-100 \cdot h - h_{\text{target}})$
Base Acceleration	$r = \exp(-3 \cdot \ \mathbf{a}_{\text{base}}\)$
Velocity Stability	$r = \frac{1}{2} [\exp(-10v_z^2) + \exp(-5 \cdot \ \omega_{xy}\)]$
Velocity Tracking (Hard)	$r = \frac{1}{2} [\exp(-10e_{\text{lin}}) + \exp(-10e_{\text{yaw}})]$
Tracking Linear Velocity	$r = \exp(-\sigma \cdot \ \mathbf{v}_{\text{cmd}} - \mathbf{v}_{\text{lin}}\ ^2)$
Tracking Angular Velocity	$r = \exp(-\sigma \cdot (\omega_{\text{cmd}} - \omega_z)^2)$
Feet Clearance	$r = \sum \mathbb{I}(h_{\text{foot}} - h_{\text{target}} < 0.01) \cdot \mathbb{I}_{\text{swing}}$
Low Speed Penalty	$r = \begin{cases} -2.0, & \text{sign mismatch} \\ -1.0, & \text{speed too low} \\ 1.2, & \text{desired speed} \\ 0.0, & \text{speed too high} \end{cases}$
Torque Penalty	$r = \sum \tau^2$
DOF Velocity Penalty	$r = \sum \dot{q}^2$
DOF Acceleration Penalty	$r = \sum \left(\frac{\dot{q}_t - \dot{q}_{t-1}}{\Delta t} \right)^2$
Collision Penalty	$r = \sum \mathbb{I}_{\ \mathbf{F}_{\text{contact}}\ > 0.1}$
Action Smoothness	$r = \ \Delta a\ ^2 + \ \Delta^2 a\ ^2 + 0.05 \cdot \ a\ _1$

To verify the generalization capability of the learned policy, we construct a multi-stage transfer pipeline. The policy is first trained in Isaac Gym using large-scale parallelized simulation. It is then validated in MuJoCo [7], which features different simulation dynamics and contact models, followed by verification in Gazebo [3], a high-fidelity robotics simulator. Finally, the policy is deployed on the real-world Dora2 bipedal platform. This progressive evaluation scheme ensures stable sim-to-real transfer and demonstrates the robustness of our method under increasingly realistic conditions.

4 Experiments and Results

To train and evaluate our policy, we leverage several physics simulators, including IsaacGym [4], MuJoCo, and Gazebo, which is shown in Fig.2. Each simulator provides complementary advantages in simulation speed, accuracy, and visual fidelity, helping

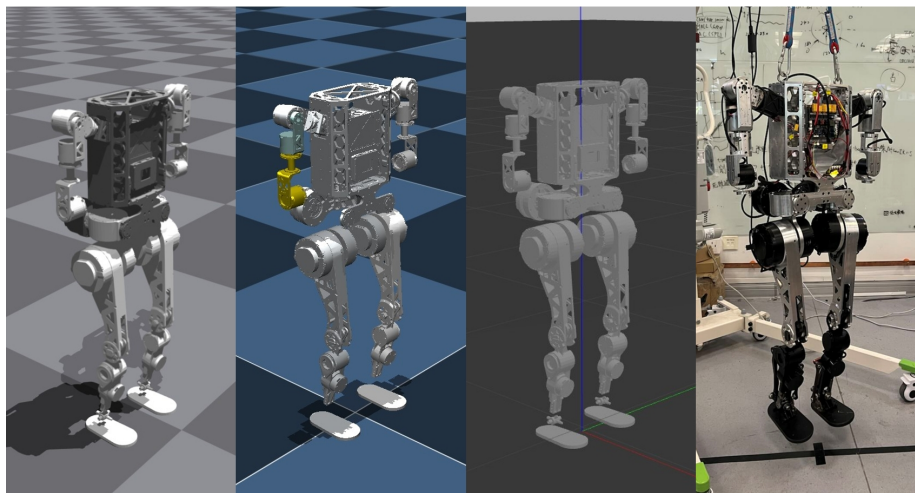


Figure 2: The Dora2 humanoid robot across platforms: IsaacGym, MuJoCo, Gazebo, and real-world deployment.

us assess policy generalization and robustness.

The training environment supports parallel simulation of multiple humanoid agents. Observations include joint positions, joint velocities, IMU data, and base orientation. The policy outputs target joint positions, which are then tracked via low-level PD controllers.

To bridge the sim-to-real gap, domain randomization is applied during training, introducing noise and variability in mass, friction, sensor delay, terrain irregularities, and external perturbations. This encourages the policy to generalize better to the physical world.

4.1 Training Performance

Our PPO-based training process shows stable convergence across multiple environments. Fig. 4 illustrates the policy and value loss decreasing over training iterations, while Fig. 3 shows the average reward steadily improving, indicating better walking performance and control stability.

Notably, after approximately 2,000 steps, the policy begins to exhibit consistent walking patterns across all simulation platforms, validating the effectiveness of the reward function design and domain randomization strategy.

4.2 Evaluation Metrics

We evaluate our policy across simulation and real-world settings using the following metrics:

- **Velocity Tracking Error:** Euclidean distance between commanded and actual

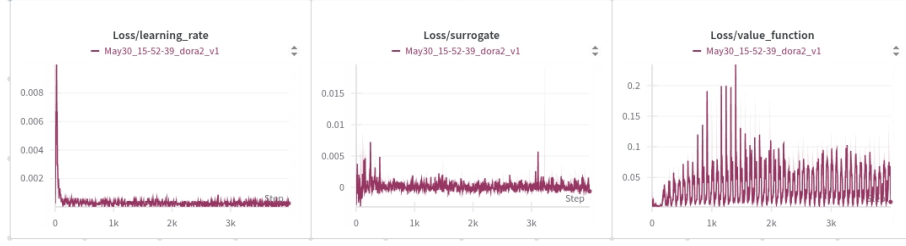


Figure 3: Episode reward curve during training, averaged across parallel environments.

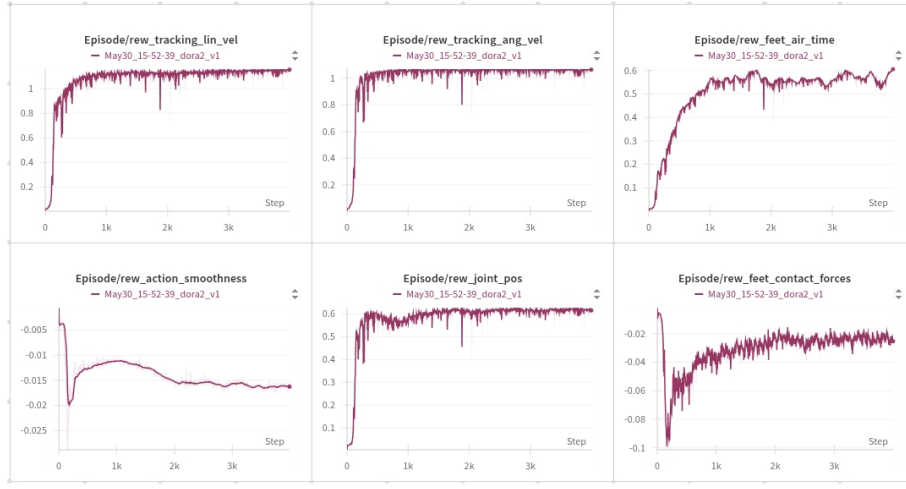


Figure 4: Training loss curve of the PPO policy over time.

linear/angular velocity, $|\mathbf{v}_{\text{cmd}} - \mathbf{v}_{\text{actual}}|_2$, across various speeds from 0.1 to 1.2 m/s.

- **Stability During Stationary Stance:** Measured by base oscillation magnitude and residual motion.
- **Observer Accuracy:** Comparison between estimated and ground-truth base velocity and foot contact states.
- **Sim-to-Sim Transfer Success Rate:** Percentage of successful walking transfers from Isaac Gym to MuJoCo.
- **Physical Deployment Fall Rate:** Ratio of trials ending in a fall on real-world hardware.

4.3 Real-World Deployment

Following training, we transfer the policy to the physical Dora2 humanoid robot. Deployment is performed with the following safety and real-time infrastructure:

- **Joint Limit and Torque Constraints:** Hardware-level safety filters prevent damage due to overextension or high torques.
- **Real-Time Monitoring:** State logs and visualizations provide live diagnostics of joint states and contact forces.
- **Policy Post-processing:** Outputs are smoothed to avoid abrupt behavior transitions.

While the policy successfully enables short sequences of walking, performance degradation is observed due to sim-to-real discrepancies. In particular, the robot tends to fall after a few steps, highlighting the need for stronger robustness and adaptation mechanisms.

5 Discussion and Future Work

In this work, we proposed an enhanced PPO-based reinforcement learning framework tailored for sim-to-real bipedal locomotion. Our method incorporates an observer module that estimates latent states such as floating-base velocity and foot contact status from raw proprioceptive data. This asymmetric observation architecture enables the policy to infer partially observable dynamics, improving both stability and generalization.

Our initial project goals were centered on three aspects: (1) enabling omnidirectional variable-speed walking via joystick input, (2) ensuring stable stationary stance without residual oscillations, and (3) achieving robust policy transfer from simulation to real-world deployment. Empirical evaluations across Isaac Gym, MuJoCo, and physical hardware confirm that these goals have been largely met. The policy exhibits smooth gait transitions, accurate velocity tracking, and consistent behavior across domains—even in the presence of sensor noise and unmodeled dynamics.

Furthermore, our modular sim-to-real transfer pipeline, which includes domain randomization and intermediate validation in MuJoCo, has proven effective in mitigating the simulation gap. The introduction of a history-conditioned observation encoder and privileged critic input further stabilizes learning and improves sample efficiency.

Future Work. While our method demonstrates strong generalization capabilities, several directions remain open for exploration:

- **Visual and Exteroceptive Sensing:** Integrating vision or LiDAR into the observation space could enable terrain-aware locomotion, obstacle avoidance, and more reactive behaviors.
- **Adaptive Terrain Control:** Extending the policy to dynamically adapt to unknown, uneven, or deformable terrains using online estimation and meta-learning techniques.

- **Energy Efficiency Optimization:** Incorporating learned reward shaping or curriculum strategies to explicitly penalize energy consumption and promote more naturalistic walking patterns.
- **General-Purpose Skill Composition:** Exploring hierarchical RL or skill primitives to enable transitions between locomotion, turning, and balancing behaviors under a unified control policy.
- **Extended Hardware Validation:** Expanding tests on more real-world conditions such as slopes, external perturbations, and long-horizon navigation tasks to further assess robustness.
- **Multi-robot collaboration:** Multiple robots observe each other to provide better observations.

We believe that this framework forms a solid foundation for future research in scalable, generalizable, and efficient learning-based locomotion controllers for humanoid robots.

References

- [1] Zhaoyuan Gu, Junheng Li, Wenlan Shen, Wenhao Yu, Zhaoming Xie, Stephen McCrory, Xianyi Cheng, Abdulaziz Shamsah, Robert Griffin, C. Karen Liu, Abderahmane Kheddar, Xue Bin Peng, Yuke Zhu, Guanya Shi, Quan Nguyen, Gordon Cheng, Huijun Gao, and Ye Zhao. Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning, 2025.
- [2] Julian Ibarz, Jie Tan, Chelsea Finn, Mrinal Kalakrishnan, Peter Pastor, and Sergey Levine. How to train your robot with deep reinforcement learning: lessons we have learned. *The International Journal of Robotics Research*, 40(4-5):698–721, 2021.
- [3] N. Koenig and A. Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, volume 3, pages 2149–2154 vol.3, 2004.
- [4] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance gpu-based physics simulation for robot learning, 2021.
- [5] Noetix Robotics. Products – noetix robotics. <https://noetixrobotics.com/products.html>, 2025. Accessed: 2025-05-04.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- [7] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.